

DSA LAB PROJECT

LIBRARY MANAGEMENT SYSTEM

Group Members :

Humayun Masood (102)
Sharjeel Shakeel (082)
Hassan Shakeel (058)

1. Introduction

In modern digital environments, efficient storage, retrieval, and processing of data are fundamental requirements. Libraries contain vast collections of books, and managing them manually is both inefficient and prone to errors. A computerized library management system eliminates these issues by enabling fast searches, systematic tracking of reading history, and intelligent analysis of book popularity.

This report presents the complete documentation for the Library Management and Recommendation System, developed as a semester project for the Data Structures and Algorithms course at Bahria University, Islamabad. The system is implemented in C++ and demonstrates the practical application of key data structures including Linked Lists, Binary Search Trees (BST), Stacks, and Priority Queues.

The system provides users with a menu-driven interface to add books, search by multiple criteria, track reading history, and discover the most viewed and top-rated titles. The project illustrates how carefully selected data structures can solve real-world problems efficiently.

2. Objectives

The primary objectives of this project are:

1. To develop a fully functional C++ system for managing library books efficiently.
 2. To implement and integrate both linear (Linked List, Stack) and non-linear (BST, Priority Queue) data structures.
 3. To demonstrate efficient searching and data retrieval techniques using tree-based and linear algorithms.
 4. To apply priority-based logic for ranking books by view count and average rating.
 5. To provide a clean, menu-based console interface for user interaction.
 6. To implement persistent file-based storage so data is saved and loaded between sessions.
-

3. System Architecture & Design

The system follows a modular, object-oriented architecture where each data structure is encapsulated in its own class. The LibrarySystem class acts as the central controller, composing all data structure objects and coordinating operations between them.

3.1 Class Structure

Class / File	Responsibility
Book.h	Defines the Book struct with all book attributes
ListNode.h	Node structure for the singly linked list
LinkedList.h / .cpp	Stores all books; supports add, find, display operations
BST.h / .cpp	Binary search tree indexed by book ID for fast lookup
Stack.h / .cpp	Session-based search history using LIFO stack
PriorityQueue.h / .cpp	Max-heap for ranking books by views or rating
FileHandler.h / .cpp	File I/O for loading/saving books to library.txt
LibrarySystem.h / .cpp	Core controller orchestrating all operations
main.cpp	Entry point; initializes and runs the system

3.2 Book Data Structure

Each book in the system stores the following attributes:

Field	Type	Description
id	int	Unique identifier for the book
title	string	Title of the book
author	string	Name of the author
genre	string	Category (Mystery, Fantasy, Adventure, etc.)
viewCount	int	Number of times the book was accessed
totalRating	float	Sum of all ratings received
ratingCount	int	Number of ratings (used to compute average)

4. Data Structures Used

4.1 Singly Linked List

The LinkedList class serves as the primary storage mechanism for all books in the library. Books are stored as dynamically allocated ListNode objects chained together via pointers.

Key characteristics:

- Dynamic memory allocation no predefined size limit.

- Each node stores a complete Book struct and a pointer to the next node.
- Supports operations: `addBook()`, `findByID()`, `displayAll()`, and `clear()`.
- Used as the backing store for linear searches by title, author, and genre.

4.2 Binary Search Tree (BST)

A Binary Search Tree is used to organise book IDs for faster retrieval when searching by ID. The BST is maintained in parallel with the linked list it does not store full book data, only the integer IDs.

Properties:

- Left subtree contains nodes with IDs smaller than the current node.
- Right subtree contains nodes with IDs greater than the current node.
- Search time complexity: $O(\log n)$ in balanced cases, versus $O(n)$ for linear search.
- Supports: `insert(id)`, `search(id)`, and `clear()` operations.

4.3 Stack

A Stack is used to maintain the session-based search history. Every time a user views a book, its ID is pushed onto the stack. The user can then display all previously viewed books in the current session.

Properties:

- Follows the Last In, First Out (LIFO) principle.
- Implemented as a linked stack using `StackNode` pointers.
- Supports: `push(id)`, `pop()`, `isEmpty()`, and `display()` operations.
- The `display()` method cross-references the `LinkedList` to show full book details.

4.4 Priority Queue (Max-Heap)

Two Priority Queue instances are maintained one for most-viewed books and one for top-rated books. Both are implemented as max-heaps using a dynamic array of `PQNode` objects.

Properties:

- Each `PQNode` stores a book ID and its priority value (view count or average rating).
- Heapify-up and heapify-down operations maintain the heap property after insertions and deletions.
- `extractMax()` retrieves the highest-priority book.
- Used to display the Top 3 Most Viewed and Top 3 Highest Rated books.

5. Module-Wise Implementation

5.1 Add Book

The `addBook()` function prompts the user to enter a Book ID, title, author, and genre. Before insertion, the BST is queried to verify the ID does not already exist, preventing duplicate entries. On success, the book is added to the LinkedList and its ID is inserted into the BST.

5.2 Search by ID

The `searchByID()` function leverages the BST to confirm the ID exists in $O(\log n)$ time, then uses `findByID()` on the LinkedList to retrieve full book details. After displaying the information, the view count is incremented, the book ID is pushed onto the history Stack, and the priority queues are rebuilt.

5.3 Search by Title

The `searchByTitle()` function performs a linear traversal of the LinkedList, performing a case-insensitive substring comparison against each book's title. Matching books are displayed, and the user may select one by ID to view full details and increment its view count.

5.4 Search by Author

Similar to title search, this function traverses the LinkedList and collects all books by the specified author. All results are presented, allowing the user to select a specific book by ID to view further details.

5.5 Search by Genre

The system supports the following genre categories: Mystery, Fantasy, Adventure, Comedy, Romance, Historical, Drama, and Classic. Users select a genre from a sub-menu, and all matching books are displayed.

5.6 Session Search History

The Stack stores a record of all books viewed during the current session. Calling `showSessionHistory()` traverses the stack and displays book titles and authors for each ID. This follows the LIFO order, showing the most recently viewed book first.

5.7 All-Time History

The FileHandler class reads from `history.txt` to display all books ever viewed across all previous sessions. This complements the in-session stack with a persistent, longer-term view history.

5.8 Most Viewed Books

The `viewPQ` priority queue is populated by inserting all books with their view counts as priorities. `extractMax()` is called up to three times to retrieve and display the top three most viewed titles.

5.9 Top Rated Books

The ratingPQ priority queue is populated with books that have received at least one rating, using their computed average rating ($\text{totalRating} / \text{ratingCount}$) as the priority. The top three are extracted and displayed.

5.10 Show All Books

The showAllBooks() function calls displayAll() on the LinkedList, presenting a formatted table of all books with their IDs, titles, view counts, and average ratings.

5.11 File Persistence

The FileHandler class handles all file I/O. On startup, loadFromFile() reads library.txt to populate the LinkedList and BST with previously saved books. On exit or after each modification, saveToFile() serialises all books back to disk, ensuring no data is lost between sessions.

6. Algorithms Used

6.1 BST Insertion & Search

Book IDs are inserted into the BST using recursive insertion. The search algorithm traverses left or right subtrees based on ID comparison, achieving $O(\log n)$ average-case time complexity.

Algorithm	Time Complexity	Use Case
BST Search	$O(\log n)$ avg, $O(n)$ worst	Search by Book ID
Linear Search	$O(n)$	Search by Title / Author / Genre
Heap Insert	$O(\log n)$	Inserting into Priority Queue
Heap Extract-Max	$O(\log n)$	Retrieving top-viewed/top-rated book
Stack Push/Pop	$O(1)$	Recording & retrieving view history
Linked List Insert	$O(1)$ at head	Adding new books

6.2 Linear Search

Searching by title, author, or genre requires a full traversal of the linked list since these fields are not indexed. Each node is visited sequentially and compared against the search term using case-insensitive matching.

6.3 Heap-Based Priority Sorting

The priority queues use a binary max-heap. On insertion, heapifyUp() ensures the new element bubbles up to its correct position. On extraction, the root (maximum element) is removed, the last element is placed at the root, and heapifyDown() restores the heap property.

7. File Structure

File	Type	Purpose
main.cpp	C++ Source	Application entry point
Book.h	Header	Book data structure definition
ListNode.h	Header	Linked list node definition
LinkedList.h / .cpp	Header + Source	Linked list implementation
BST.h / .cpp	Header + Source	Binary search tree implementation
Stack.h / .cpp	Header + Source	Stack implementation
PriorityQueue.h / .cpp	Header + Source	Priority queue (max-heap) implementation
LibrarySystem.h / .cpp	Header + Source	Main system controller
FileHandler.h / .cpp	Header + Source	File I/O operations
library.txt	Data File	Persistent book storage
history.txt	Data File	Persistent view history log

8. System Workflow

The following steps describe the end-to-end flow of the system:

7. Program starts; LibrarySystem constructor is called.
8. FileHandler loads books from library.txt into the LinkedList and BST.
9. The main menu is displayed with all available options.
10. User selects an operation (e.g., Search by ID).
11. The appropriate method is invoked, utilising the relevant data structure.
12. For book views: view count is incremented, ID is pushed to the Stack, and priority queues are updated.
13. For ratings: user provides a score (1–5); totalRating and ratingCount are updated.
14. After each modification, saveToFile() persists the updated state to disk.
15. User may continue interacting until they choose to exit.
16. On exit, all in-memory data is saved and the program terminates cleanly.

9. Sample Output / Testing

9.1 Add Book

Test: Adding a new book with ID 101, Title "The Great Gatsby", Author "F. Scott Fitzgerald", Genre "Classic".

Expected: System checks BST ID not found. Book is added to LinkedList and BST. Confirmation message displayed.

```
Enter Book ID: 101
Enter Title: The Great Gatsby
Enter Author: F. Scott Fitzgerald
Enter Genre: Classic
Book added successfully!
```

9.2 Search by ID

Test: Searching for Book ID 101.

```
Enter Book ID to search: 101
ID: 101 | Title: The Great Gatsby | Author: F. Scott Fitzgerald
Genre: Classic | Views: 1 | Average Rating: 0.0
Would you like to rate this book? (1-5): 4
Rating saved. Thank you!
```

9.3 Most Viewed & Top Rated

After several interactions, the Most Viewed and Top Rated features correctly extract the top 3 entries from each priority queue, validating the heap-based ranking mechanism.

10. Challenges Faced

Challenge	Solution Applied
Maintaining BST and LinkedList in sync	Every add/remove operation updates both structures atomically in LibrarySystem methods.
Rebuilding priority queues efficiently	PQs are cleared and repopulated after each view/rating update to reflect current data.
Persistent storage across sessions	FileHandler serialises and deserialises Book structs to/from a plain-text file format.
Duplicate book ID prevention	BST search is performed before every insert; duplicate IDs are rejected with an error message.
Case-insensitive text search	Title and author comparisons convert both strings to lowercase before comparison.

11. Conclusion

This project successfully demonstrates how fundamental data structures and algorithms can be combined to build a practical, real-world application. The Library Management and Recommendation System effectively addresses the problem of managing a large collection of books using efficient computational techniques.

The Linked List provides flexible, dynamic storage for all book records. The Binary Search Tree enables fast ID-based retrieval with logarithmic search performance. The Stack offers a natural, intuitive mechanism for tracking viewing history in LIFO order. The Priority Queue delivers intelligent ranking of books by popularity and user ratings.

Together, these structures demonstrate the importance of selecting the right data structure for each specific task. The modular design also ensures that individual components can be tested, replaced, or extended independently, making the system maintainable and scalable.

Future enhancements could include implementing a balanced BST (AVL or Red-Black Tree) for guaranteed $O(\log n)$ performance, adding a graphical user interface, or integrating a database backend for larger-scale deployment.